

ANN and DL (CAI-361)

Objectives:

- Back Propagation neural Network
 - Gradient Descent
 - Types of losses
-

Feed Forward Neural Networks

The feedforward neural network was the first and simplest type of artificial neural network devised. In this network, the information moves in only one direction—forward—from the input nodes, through the hidden nodes (if any) and to the output nodes. There are no cycles or loops in the network.

Back Propagation Neural Network:

Backpropagation is a widely used algorithm for training feedforward neural networks. It computes the gradient of the loss function with respect to the network weights.

Steps in Back propagation Neural Network (BNN) are

Step 1: Inputs X, arrive through the preconnected path.

Step 2: The input is modeled using true weights W. Weights are usually chosen randomly.

Step 3: Calculate the output of each neuron from the input layer to the hidden layer to the output layer.

Step 4: Calculate the error in the outputs

$$\text{Backpropagation Error} = \text{Actual Output} - \text{Desired Output}$$

Step 5: From the output layer, go back to the hidden layer to adjust the weights to reduce the error.

Step 6: Repeat the process until the desired output is achieved.

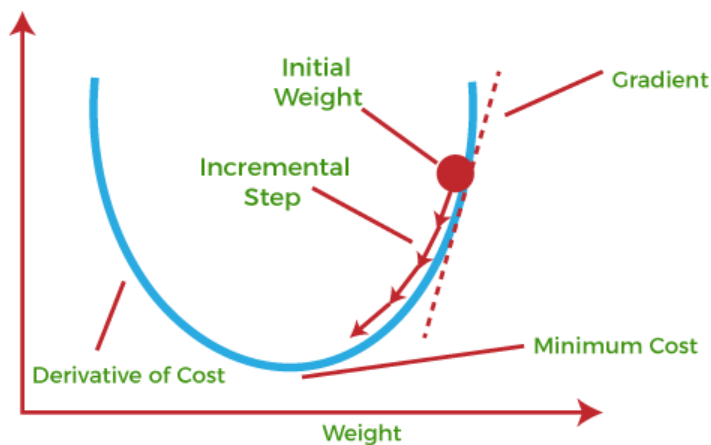
Gradient Descent:

Gradient descent (GD) is an iterative first-order optimization algorithm used to find a local minimum/maximum of a given function. This method is commonly used in *machine learning* (ML) and *deep learning* (DL) to minimize a cost/loss function.

In mathematical terminology, Optimization algorithm refers to the task of minimizing/maximizing an objective function $f(x)$ parameterized by x . Similarly, in machine learning, optimization is the task of minimizing the cost function parameterized by the model's parameters.

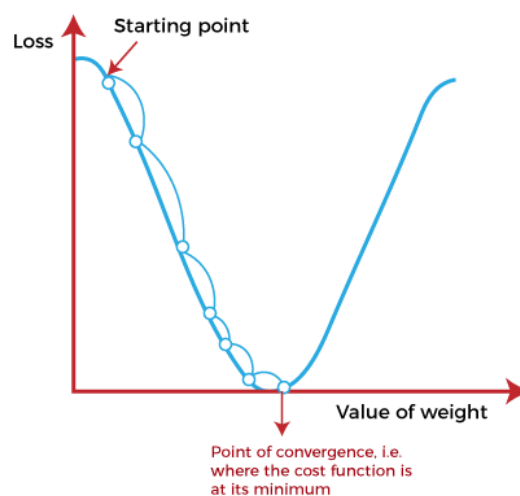
The best way to define the local minimum or local maximum of a function using gradient descent is as follows:

- If we move towards a negative gradient or away from the gradient of the function at the current point, it will give the **local minimum** of that function.
- Whenever we move towards a positive gradient or towards the gradient of the function at the current point, we will get the **local maximum** of that function.



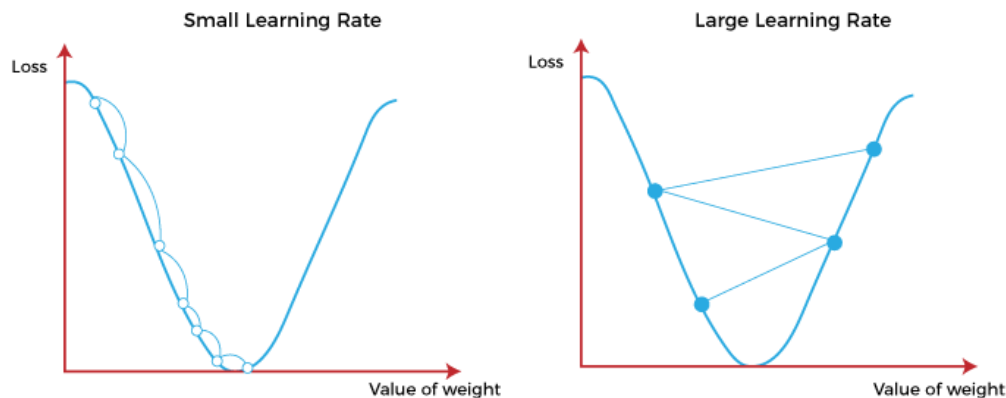
This entire procedure is known as Gradient Ascent, which is also known as steepest descent. ***The main objective of using a gradient descent algorithm is to minimize the cost function using iteration.*** To achieve this goal, it performs two steps iteratively:

- Calculates the first-order derivative of the function to compute the gradient or slope of that function.
- Move away from the direction of the gradient, which means slope increased from the current point by alpha times, where Alpha is defined as Learning Rate. It is a tuning parameter in the optimization process which helps to decide the length of the steps.



Learning Rate:

It is defined as the step size taken to reach the minimum or lowest point. This is typically a small value that is evaluated and updated based on the behavior of the cost function. If the learning rate is high, it results in larger steps but also leads to risks of overshooting the minimum. At the same time, a low learning rate shows the small step sizes, which compromises overall efficiency but gives the advantage of more precision.



Types of Gradient Descents:

1. Batch Gradient Descent:

Batch gradient descent (BGD) is used to find the error for each point in the training set and update the model after evaluating all training examples. This procedure is known as the training epoch. In simple words, it is a greedy approach where we have to sum over all examples for each update.

2. Stochastic gradient descent

Stochastic gradient descent (SGD) is a type of gradient descent that runs one training example per iteration. Or in other words, it processes a training epoch for each example within a dataset and updates each training example's parameters one at a time. As it requires only one training example at a time, hence it is easier to store in allocated memory. However, it shows some computational efficiency losses in comparison to batch gradient systems as it shows frequent updates that require more detail and speed. Further, due to frequent updates, it is also treated as a noisy gradient. However, sometimes it can be helpful in finding the global minimum and also escaping the local minimum.

3. MiniBatch Gradient Descent:

Mini Batch gradient descent is the combination of both batch gradient descent and stochastic gradient descent. It divides the training datasets into small batch sizes then performs the updates on those batches separately. Splitting training datasets into smaller batches make a balance to maintain the computational efficiency of batch gradient descent and speed of stochastic gradient descent. Hence, we can achieve a special type of gradient descent with higher computational efficiency and less noisy gradient descent.

Challenges with gradient descent

1. Local minima and saddle points

Recall that when the slope of the cost function is at or close to zero, the model stops learning. A few scenarios beyond the global minimum can also yield this slope, which are local minima and saddle points. Local minima mimic the shape of a global minimum, where the slope of the cost function increases on either side of the current point. However, with saddle points, the negative gradient only exists on one side of the point, reaching a local maximum on one side and a local minimum on the other. Its name inspired by that of a horse's saddle.

2. Vanishing and Exploding gradients

- **Vanishing gradients:** This occurs when the gradient is too small. As we move backwards during backpropagation, the gradient continues to become smaller, causing the earlier layers in the network to learn more slowly than later layers. When this happens, the weight parameters update until they become insignificant—i.e. 0—resulting in an algorithm that is no longer learning.
- **Exploding gradients:** This happens when the gradient is too large, creating an unstable model. In this case, the model weights will grow too large, and they will eventually be represented as NaN. One solution to this issue is to leverage a dimensionality reduction technique, which can help to minimize complexity within the model.

Types of Losses:

Mean Absolute Error (MAE)

Mean absolute error (MAE) also called **L1 Loss** is a loss function used for regression problems. It represents the difference between the original and predicted values extracted by averaging the absolute difference over the data set.

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}|$$

MAE is *not sensitive towards outliers* and is given several examples with the same input feature values, and the optimal prediction will be their median target value. This should be compared with Mean Squared Error, where the optimal prediction is the mean. A disadvantage of MAE is that the gradient magnitude is not dependent on the error size, only on the sign of $y - \hat{y}$ which leads to that the gradient magnitude will be large even when the error is small, which in turn can lead to convergence problems.

Mean Squared Error (MSE)

Mean Squared Error (MSE) also called **L2 Loss** is also a loss function used for regression. It represents the difference between the original and predicted values extracted by squared the average difference over the data set.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y})^2$$

MSE is *sensitive towards outliers* and given several examples with the same input feature values, the optimal prediction will be their mean target value. This should be compared with Mean Absolute Error, where the optimal prediction is the median. MSE is thus good to use if you believe that your target data, conditioned on the input, is normally distributed around a mean value, and when it's important to penalize outliers extra much.

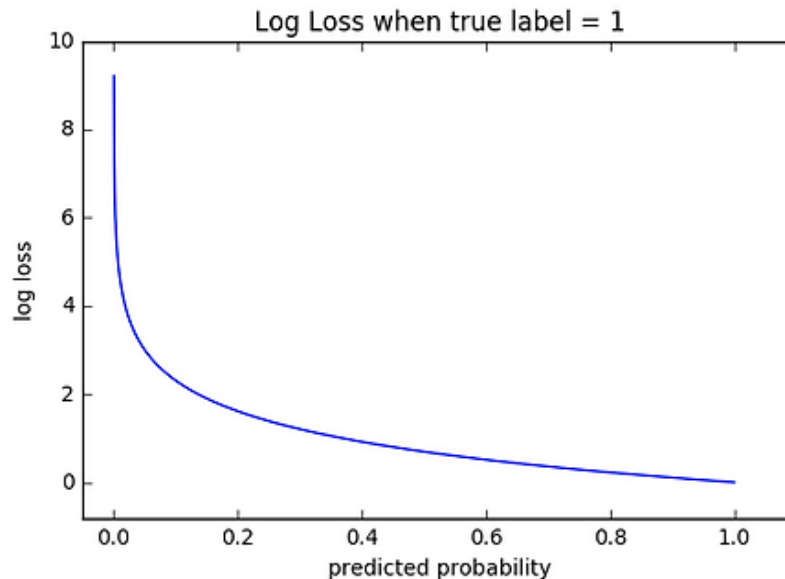
Cross-Entropy Loss (Log loss)

Cross-Entropy loss is also called **logarithmic loss**, **log loss**, or **logistic loss**. Each predicted class probability is compared to the actual class desired output 0 or 1 and a score/loss is calculated that penalizes the probability based on how far it is from the actual expected value. The penalty is logarithmic in nature yielding a large score for large differences close to 1 and small score for small differences tending to 0. Cross-Entropy is expressed by the equation;

$$I(p, q) = -\sum p(x) \log q(x)$$

Where $\mathbf{x}$ represents the predicted results by ML algorithm, $\mathbf{p}(\mathbf{x})$ is the probability distribution of “true” label from training samples and $\mathbf{q}(\mathbf{x})$ depicts the estimation of the ML algorithm.

Cross-entropy loss measures the performance of a classification model whose output is a probability value between 0 and 1. Cross-entropy loss increases as the predicted probability diverge from the actual label. So predicting a probability of .012 when the actual observation label is 1 would be bad and result in a high loss value. A perfect model would have a log loss of 0.



Categorical Cross-Entropy

Categorical cross-entropy is a loss function that is used in multi-class classification tasks. These are tasks where an example can only belong to one out of many possible categories, and the model must decide which one.

Formally, it is designed to quantify the difference between two probability distributions.

If $M > 2$ (i.e. multiclass classification), we calculate a separate loss for each class label per observation and sum the result.

$$-\sum_{c=1}^M y_{o,c} \log(p_{o,c})$$

- M —number of classes (dog, cat, fish)
- $\log$ —the natural log
- y —binary indicator (0 or 1) if class label c is the correct classification for observation o
- p —predicted probability observation o is of class c

Softmax is the only activation function recommended to use with the categorical cross-entropy loss function. Strictly speaking, the output of the model only needs to be positive so that the logarithm of every output value $\hat{y}_i$ exists. However, the main appeal of this loss function is for comparing two probability distributions. The softmax activation rescales the model output so that it has the right properties.

Practice Task1:

The class and predicted columns in the given dataset have a string to represent each class while to write the loss function we need to use a number what strategy do you think is appropriate for class and predicted columns for the loss function? Also, write the loss function for the table and give a numerical value for the loss function.

Qualification	Experience	Job Sector	Designation	Salary	Classes	Predicted
Graduation	5	Petroleum	Site Engineer	5,30,000	excellent	excellent
Post Graduation	2	Telecommunication	Site Support Engineer	85,000	excellent	appropriate
Intermediate	1	Telecommunication	Clerk	20,000	below average	below average
Matriculation	6	Education	Guard	25,000	appropriate	appropriate
Intermediate	15	Banking	Casher	65,000	excellent	appropriate
Post Graduation	12	Education	Associate Professor	3,50,000	excellent	excellent
Graduation	10	Telecommunication	Manager	2,50,000	appropriate	below average
Graduation	12	Education	Assistant Professor	2,10,000	below average	appropriate

Practice Task2:

Use gradient descent to find the **new values of θ_1 and θ_2** assume initial value of θ_1 to be 0.05 θ_2 to be 0.10, and $\eta = 0.01$. Show two steps of gradient descent, in second step use the values of θ_1 and θ_2 obtained from the first step.

x_1	y	$\hat{y}$
0.5	1	1
0.2	0	1
0.3	0	0
0.7	1	0
0.9	1	1